

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ

Федеральное государственное автономное
образовательное учреждение высшего образования
«Национальный исследовательский университет ИТМО»

Курсовой проект
по дисциплине
«Методы машинного обучения в робототехнике»

Тема: «Интерактивное управление роботом с помощью жестов руки»

Подготовили: ст. гр.

R4233с и R4236с

Стадников Д.И.

Кулить Г.В.

Проверил: д.т.н., профессор

Колюбин С.А.

Санкт-Петербург

2025 г.

Аннотация

В данной работе разработана система управления движением робота с использованием жестов, основанная на методах машинного обучения и симуляции. Основной целью исследования стало создание классификационной модели на базе сверточной нейронной сети (CNN) и фреймворка MediaPipe для распознавания жестов и интеграция этой модели с симуляцией движения робота, а также проведение сравнительного анализа двух методов.

В рамках исследования выполнены следующие этапы: анализ и предобработка готового датасета, обучение модели на подготовленных данных с использованием аугментации, разработка алгоритма управления роботом и создание симуляционной среды. Полученные результаты показали точность классификации модели на уровне 99%, что позволяет эффективно распознавать основные жесты. Симуляция движения робота подтвердила корректность выполнения команд, считываемые с кадров веб-камеры.

Разработанная система демонстрирует высокий потенциал для использования в задачах управления роботами, взаимодействия с пользователем и виртуальной реальности. Однако дальнейшая работа будет направлена на повышение устойчивости модели к реальным условиям эксплуатации, улучшение точности классификации сложных жестов и адаптацию системы для работы с реальными роботами.

Работа представляет интерес для специалистов в области робототехники, машинного обучения и человеко-машинного взаимодействия.

Содержание

Введение.....	4
1 Анализ и постановка проблемы.....	5
1.1 Актуальность темы	5
1.2 Цель и задачи работы	6
2 Обзор современных методов и подходов к решению задачи.....	7
2.1 Методы управления роботами.....	7
2.2 Классификация жестов и использование CNN	7
2.3 Фреймворк MediaPipe.....	8
3 Реализация рассматриваемых методов	9
3.1 Подготовка данных: сбор, разметка и предобработка	9
3.2 Обучение CNN для классификации жестов	9
3.3 Анализ принципа работы MediaPipe.....	12
4 Описание и анализ полученных результатов	13
4.1 Тестирование метода через собственную модель	13
4.2 Тестирование метода через MediaPipe.....	15
4.3 Сравнительный анализ двух методов	17
Заключение	20
Список литературы	22
Приложение А	23
Приложение Б	26

Введение

С развитием технологий человеко-машинного взаимодействия (HCI) управление устройствами с помощью жестов становится все более популярным и востребованным. Такие системы находят применение в робототехнике, виртуальной реальности, медицине, образовании и других областях. Управление жестами позволяет создать интуитивные и естественные интерфейсы, которые значительно упрощают взаимодействие с машинами, делая его быстрым и удобным.

В данной работе рассматривается задача управления роботом с использованием жестов, где жесты распознаются с помощью методов машинного обучения. Для распознавания жестов разработана классификационная модель на основе сверточной нейронной сети (CNN), а также использование фреймворка MediaPipe, выполнение команд осуществляется через симуляцию движения робота на 2D гриде. Такой подход позволяет объединить преимущества машинного зрения и алгоритмов управления роботами.

Разработка систем управления жестами является актуальной задачей в контексте повышения удобства и эффективности взаимодействия человека и машин. Традиционные методы управления (например, через кнопки или контроллеры) имеют ограничения, связанные с их сложностью, неудобством использования и необходимостью физического взаимодействия. Системы управления жестами решают эти проблемы, предоставляя бесконтактный и адаптивный способ взаимодействия.

Кроме того, использование методов машинного обучения, таких как CNN, позволяет создавать высокоэффективные модели для распознавания сложных паттернов, таких как жесты. Это делает системы более точными и надежными даже в условиях изменчивой окружающей среды.

1 Анализ и постановка проблемы

1.1 Актуальность темы

Современные технологии компьютерного зрения и машинного обучения играют все более значимую роль в автоматизации процессов и взаимодействии человека с машинами. Управление жестами — одно из наиболее перспективных направлений, которое позволяет создать естественный и интуитивно понятный способ взаимодействия с роботами и устройствами. Применение таких технологий находит отклик в широком спектре областей, включая:

Промышленность: управление роботами в сложных условиях, где использование традиционных методов ввода (например, клавиатуры или джойстика) затруднено или невозможно.

Здравоохранение: помощь людям с ограниченными возможностями, предоставляя им новые способы взаимодействия с окружающей средой.

Развлечения и виртуальная реальность: создание интерактивных приложений, основанных на движении и жестах.

Образование: разработка инновационных методов обучения с использованием жестов для управления обучающими системами или симуляциями.

В последние годы значительный прогресс достигнут в области компьютерного зрения благодаря разработке глубоких нейронных сетей. Однако интеграция этих технологий в пользовательские интерфейсы все еще представляет сложность, требуя точного распознавания жестов, высокой производительности в реальном времени и доступности на устройствах с ограниченными вычислительными ресурсами. Использование современных фреймворков, таких как MediaPipe, позволяет существенно упростить процесс разработки, но требует тщательной проработки архитектуры системы и обучающих моделей.

Предложенная работа представляет собой решение важной задачи: создание системы управления роботом с использованием жестов на базе классификационной модели, обученной с помощью сверточных нейронных сетей (CNN), и интеграция этой модели в симуляцию движения робота на 2D-гриде. Такая система может найти применение в многочисленных реальных сценариях, таких как удаленное управление, образовательные симуляции и адаптивные пользовательские интерфейсы.

1.2 Цель и задачи работы

Цель данной работы состоит в разработке системы управления роботом с помощью жестов, включающей обучение классификационной модели для распознавания жестов и симуляцию движения робота на 2D гриде.

Для достижения поставленной цели необходимо выполнить следующие задачи:

- Найти и пред обработать набор данных для обучения модели
- Разработать и обучить сверточную нейронную сеть для распознавания жестов.
- Создать симуляцию движения робота на 2D гриде, обеспечивающую выполнение команд, поступающих от модели классификации и фреймворка MediaPipe
- Интегрировать классификационную модель и симуляцию.
- Провести тестирование системы и проанализировать её результаты.
- Провести сравнительный анализ двух методов

2 Обзор современных методов и подходов к решению задачи

2.1 Методы управления роботами

Различные методы управления роботами находят применение в зависимости от задач и условий эксплуатации. Традиционные подходы включают:

- Клавиатурное и джойстиковое управление — наиболее распространенные методы ввода команд для роботов, используемые в промышленных и бытовых приложениях.
- Дистанционное управление с помощью радиосигналов или Wi-Fi — применяется для выполнения задач в опасных или труднодоступных условиях.
- Современные технологии управления ориентированы на повышение удобства взаимодействия:
- Голосовое управление позволяет пользователю отдавать команды роботу в естественной форме, но требует устойчивости к шуму и вариациям речи.

Управление жестами, благодаря своей интуитивности, становится все более востребованным. Оно особенно актуально в ситуациях, где голосовой ввод может быть затруднен, например, в шумной среде.

2.2 Классификация жестов и использование CNN

Распознавание жестов — это задача классификации, которая включает в себя обработку и анализ изображений или видео.

Ранние подходы: классические методы на основе обработки признаков, таких как SIFT, HOG или Haar-признаки. Они демонстрировали ограниченную точность при сложных условиях (например, изменении освещения или угла камеры) [1].

Современные методы: использование глубоких нейронных сетей, таких как CNN, позволяет обрабатывать изображения напрямую, автоматически выделяя важные признаки. CNN имеют многослойную архитектуру, включая

сверточные и пулинговые слои, которые хорошо подходят для анализа визуальных данных [2]. Существуют примеры успешных применений этих методов такие как классификация жестов для управления беспилотниками, распознавание языка жестов для людей с ограничениями слуха.

2.3 Фреймворк MediaPipe

MediaPipe — это мощный и универсальный фреймворк с открытым исходным кодом, разработанный компанией Google, который предназначен для создания решений в области обработки мультимедиа. MediaPipe предоставляет инструменты и готовые модули для анализа изображений, видео, аудио и других данных в реальном времени. Фреймворк широко используется для задач компьютерного зрения и обработки сигналов, таких как распознавание лиц, жестов, движений тела, и предлагает высокую производительность благодаря использованию современных методов машинного обучения [3].

MediaPipe предлагает множество готовых решений, которые можно использовать в проектах, включая:

Распознавание рук (Hand Tracking):

Технология позволяет отслеживать положение рук и определять ключевые точки (landmarks), такие как суставы пальцев и ладонь. Этот модуль идеально подходит для задач управления жестами [4].

Распознавание лица (Face Detection и Face Mesh):

Позволяет обнаруживать лицо на изображении, выделять ключевые точки, такие как глаза, нос, рот, и даже создавать 3D-модель лица.

Распознавание позы тела (Pose Tracking):

Анализирует движения тела и выделяет ключевые точки на основе положения суставов, что полезно для задач анализа движений.

Распознавание объектов (Object Detection):

Автоматически обнаруживает и классифицирует объекты в кадре с использованием моделей машинного обучения.

3 Реализация рассматриваемых методов

3.1 Подготовка данных: сбор, разметка и предобработка

Для создания качественного набора данных для обучения классификационной модели необходимо выполнить несколько ключевых шагов:

Сбор данных - в качестве данных был взят готовый, размеченный датасет жестов, взятый с *Kaggle* - Hand Gesture Recognition Database от LeapMotion [5].

Предобработка данных включала следующие шаги: изменение размера изображений: Все изображения были приведены к единому размеру, значения пикселей были нормализованы в диапазон $[0, 1]$ для ускорения обучения нейронной сети.

Аугментация данных: Для увеличения объема тренировочного набора использовалась аугментация, включающая случайные вращения, изменения масштаба, смещения изображений.

3.2 Обучение CNN для классификации жестов

Для создания классификационной модели использовалась архитектура сверточной нейронной сети, которая оптимально подходит для анализа изображений.

Модель была построена с использованием следующих слоев: Сверточные слои (Conv2D) с ядрами размером 3×3 , пулинг (MaxPooling) для уменьшения пространственного размера изображения, полносвязные слои (Dense) для классификации.

Функция активации ReLU для сверточных слоев и softmax для выходного слоя, так как задача является многоклассовой классификацией.

Гиперпараметры: learning rate - 0.001, размер батча (batch size) - 128, количество эпох – 3.

Для оценки модели использовались следующие метрики:

Точность (accuracy): процент правильно классифицированных изображений.

Precision, Recall, F1-score: для оценки качества модели по каждому классу жестов.

Для тренировки модели использовалась библиотека TensorFlow с Keras, что позволило ускорить процесс разработки и тестирования.

Общее кол-во обучаемых параметров: 15,617,674

Слои модели показаны на рисунке 3.1.

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 73, 73, 128)	24,704
batch_normalization (BatchNormalization)	(None, 73, 73, 128)	512
conv2d_1 (Conv2D)	(None, 73, 73, 256)	819,456
batch_normalization_1 (BatchNormalization)	(None, 73, 73, 256)	1,024
max_pooling2d (MaxPooling2D)	(None, 24, 24, 256)	0
conv2d_2 (Conv2D)	(None, 24, 24, 256)	590,880
batch_normalization_2 (BatchNormalization)	(None, 24, 24, 256)	1,024
conv2d_3 (Conv2D)	(None, 24, 24, 256)	65,792
batch_normalization_3 (BatchNormalization)	(None, 24, 24, 256)	1,024
conv2d_4 (Conv2D)	(None, 24, 24, 256)	65,792
batch_normalization_4 (BatchNormalization)	(None, 24, 24, 256)	1,024
conv2d_5 (Conv2D)	(None, 24, 24, 512)	1,180,160
batch_normalization_5 (BatchNormalization)	(None, 24, 24, 512)	2,048
max_pooling2d_1 (MaxPooling2D)	(None, 12, 12, 512)	0
conv2d_6 (Conv2D)	(None, 12, 12, 512)	2,359,808
batch_normalization_6 (BatchNormalization)	(None, 12, 12, 512)	2,048
conv2d_7 (Conv2D)	(None, 12, 12, 512)	2,359,808
batch_normalization_7 (BatchNormalization)	(None, 12, 12, 512)	2,048
max_pooling2d_2 (MaxPooling2D)	(None, 6, 6, 512)	0
conv2d_8 (Conv2D)	(None, 6, 6, 512)	2,359,808
batch_normalization_8 (BatchNormalization)	(None, 6, 6, 512)	2,048
max_pooling2d_3 (MaxPooling2D)	(None, 3, 3, 512)	0
flatten (Flatten)	(None, 4608)	0
dense (Dense)	(None, 1024)	4,719,616
dropout (Dropout)	(None, 1024)	0
dense_1 (Dense)	(None, 1024)	1,049,600
dropout_1 (Dropout)	(None, 1024)	0
dense_2 (Dense)	(None, 10)	10,250

Рисунок 3.1 – Слои модели

Обучение модели привело к следующим показателям наших метрик:

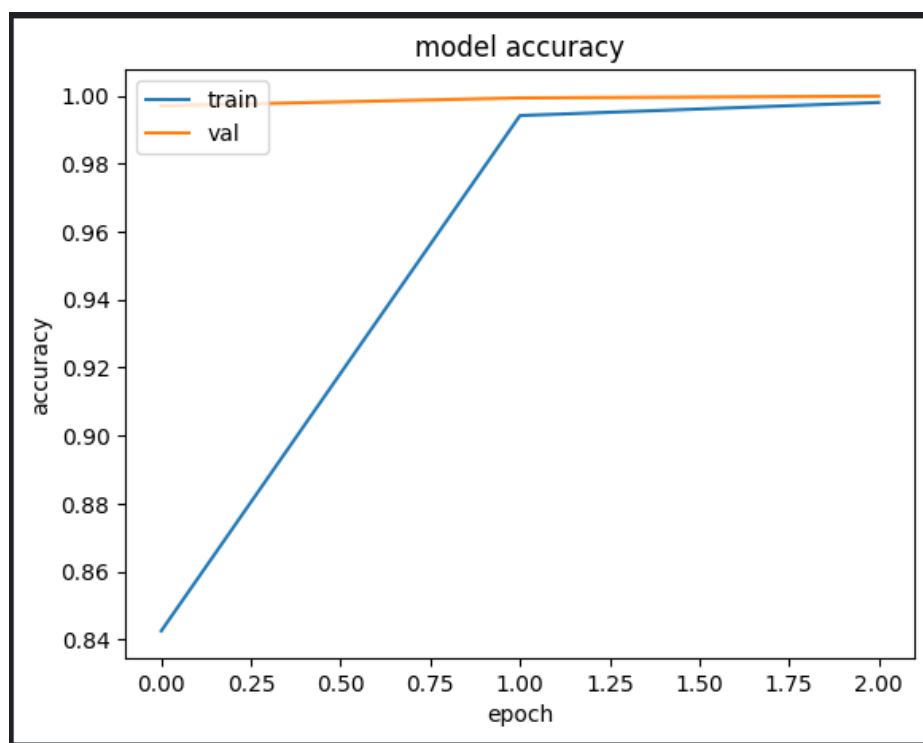


Рисунок 3.2 – Точность обученной модели

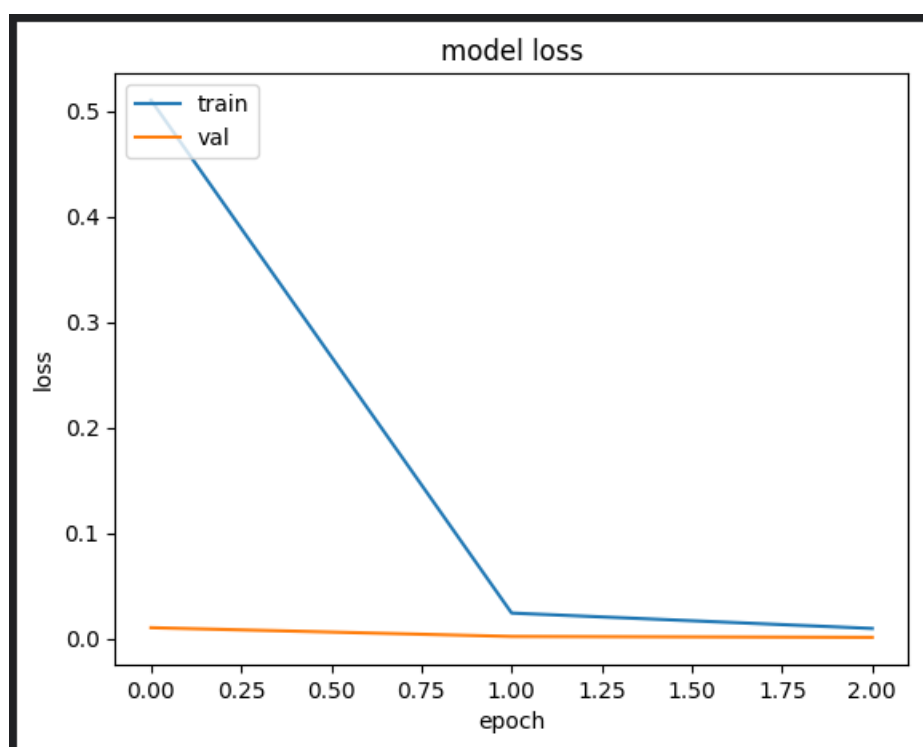


Рисунок 3.3 – Функция потерь обученной модели

Accuracy – 0.99, precision/recall/F1-score – 0.98, loss – 0.015

3.3 Анализ принципа работы MediaPipe

Используется модель Palm Detection, которая обучена находить ладони на изображениях.

Эта модель основана на сверточной нейронной сети (CNN) и оптимизирована для быстрого обнаружения ладоней.

Модель выполняет поиск областей изображения, которые могут содержать руку, определение координат bounding box, обнаружение ладоней работает с высокой скоростью благодаря использованию эффективных архитектур, таких как MobileNet, которая использует Depthwise Separable Convolutions [6]. Palm Detection фокусируется только на ладонях, что исключает ложные срабатывания для других частей тела.

После обнаружения ладони область, ограниченная bounding box, передается в модель для анализа ключевых точек.

Модель выделяет 21 ключевую точку, которые включают основание ладони, суставы каждого пальца (по четыре точки на палец), кончики пальцев.

Каждая точка имеет координаты (x, y, z), где: x и y — координаты в плоскости изображения, z — относительная глубина точки (положение точки относительно центра ладони).

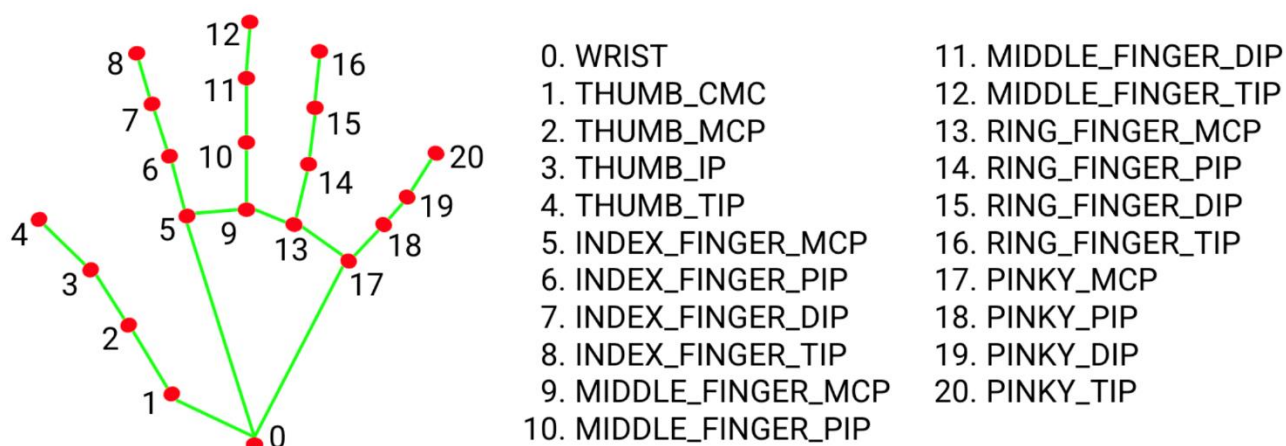


Рисунок 3.4 – Расположение ключевых точек ладони в MediaPipe

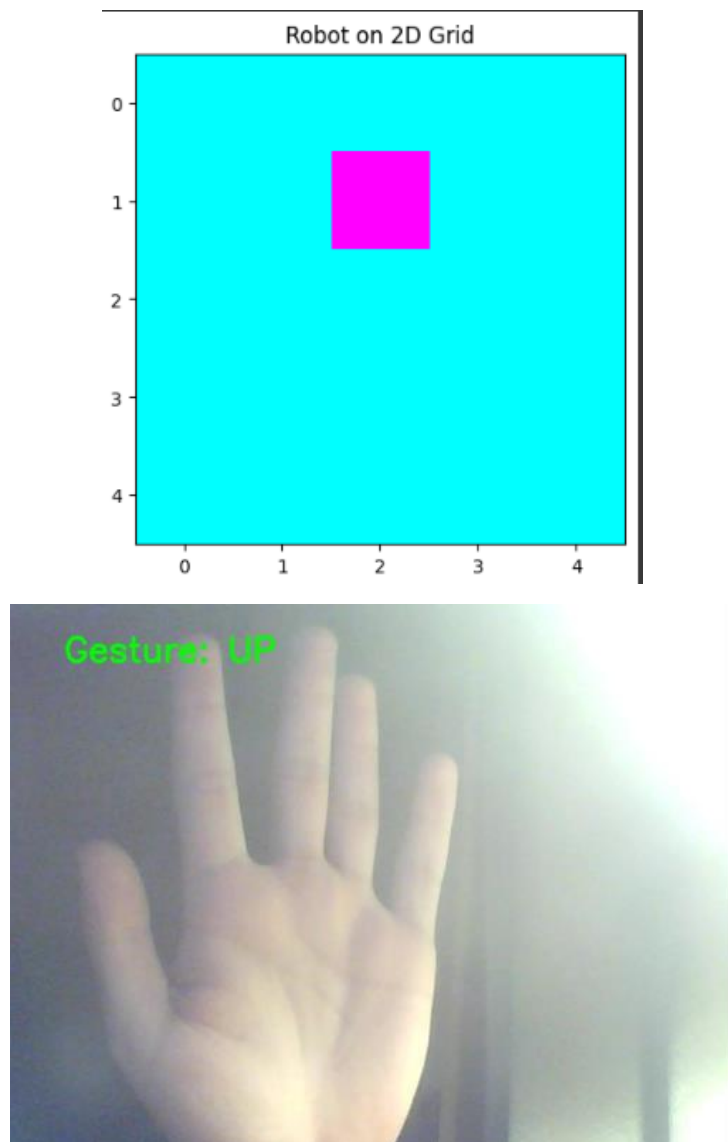
4 Описание и анализ полученных результатов

4.1 Тестирование метода через собственную модель

Для моделирования движения робота на 2D гриде использовалась библиотека matplotlib

Алгоритм обработки команд:

Каждая команда, полученная от модели классификации (например, «вперед», «вправо»), приводила к перемещению робота на одну клетку в соответствующем направлении. При запуске команды вводится желаемое количество шагов, затем веб-камера фиксирует жест на каждый шаг. Выполнение команд показано на рисунке 4.1.



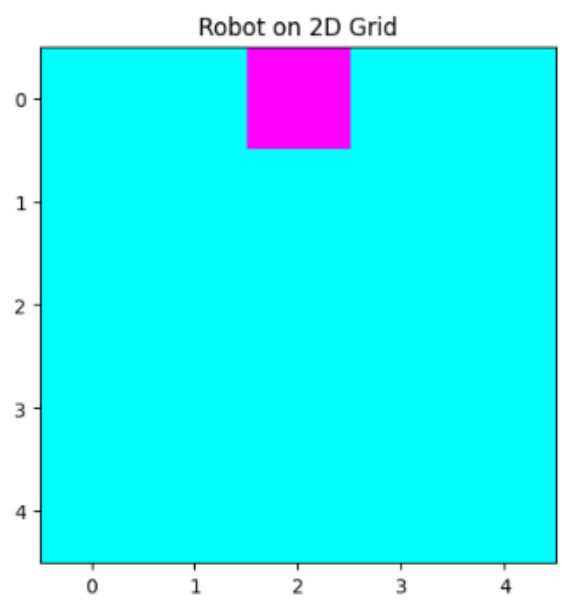


Рисунок 4.1 – Движение робота с помощью собственной модели

Как видно на рисунках модель не очень хорошо справляется на новых данных, не смотря на очень высокие показатели на обучении, такой результат может вызывать отличие качества, освещения на камере, когда модель приспособилась к изображениям первоначального датасета.

4.2 Тестирование метода через MediaPipe

На рисунках продемонстрирована работа симуляции движения робота с помощью MediaPipe

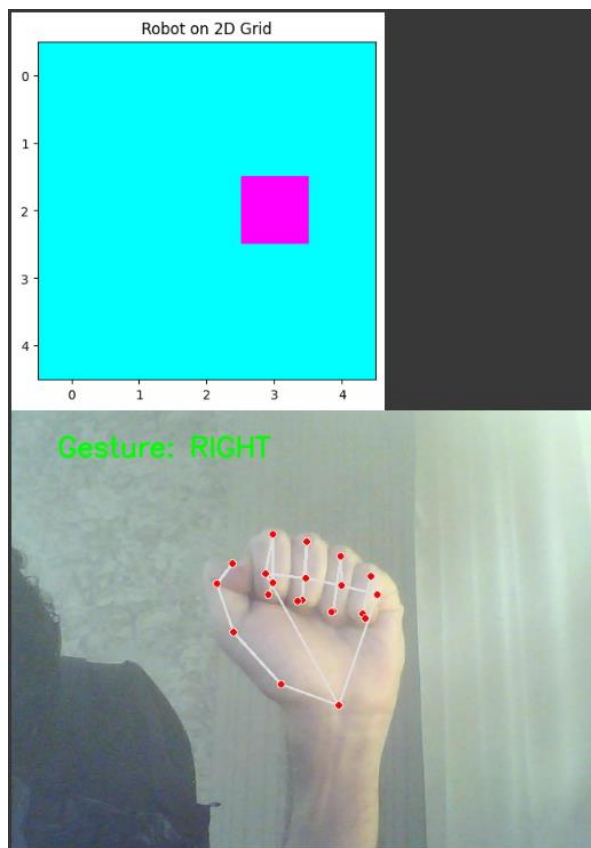


Рисунок 4.2 – Движение вправо с помощью MediaPipe

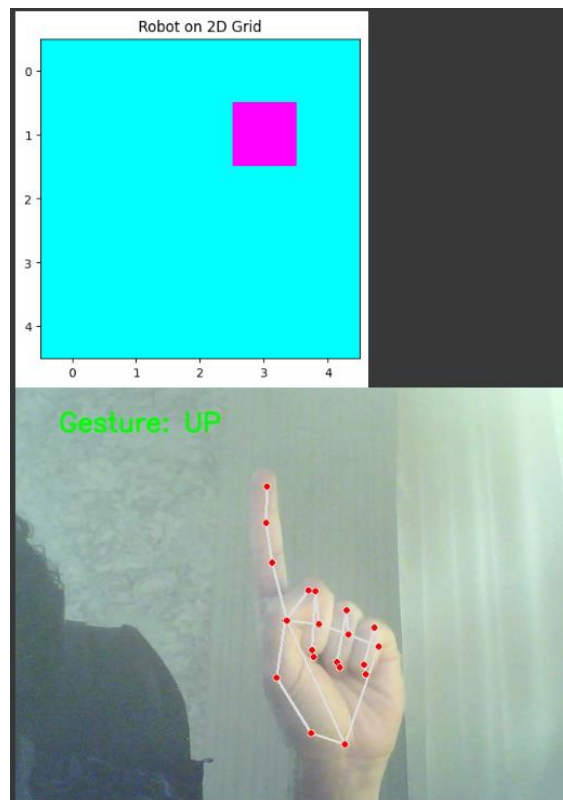


Рисунок 4.3 – Движение вверх с помощью MediaPipe

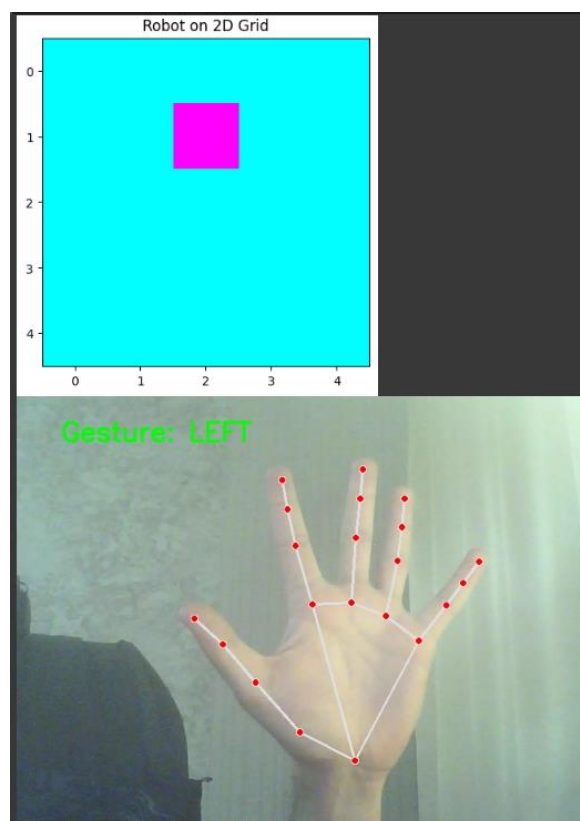


Рисунок 4.4 – Движение влево с помощью MediaPipe

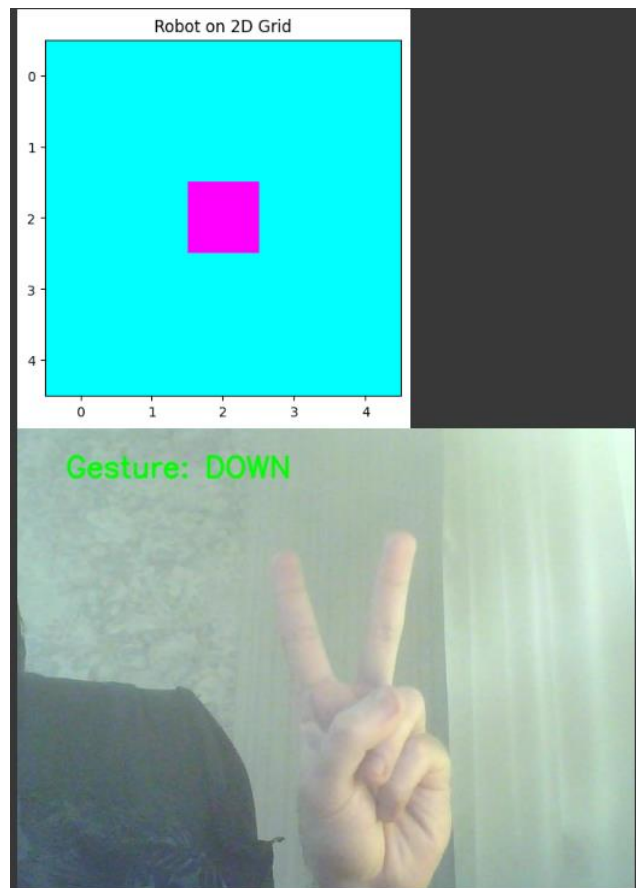


Рисунок 4.5 – Движение вниз с помощью MediaPipe

Метод распознавания с помощью MediaPipe показал себя намного лучше. Положения жестов очень точно определяются, робот движется так, как мы это задумываем.

4.3 Сравнительный анализ двух методов

Сравнительный анализ проводился на наборе из 100 жестовых команд в условиях реального времени. Точность MediaPipe оказалась выше, чем у собственной CNN. В данной главе рассмотрим детали эксперимента, причины различий в точности и сильные стороны каждого из методов.

Сравнительный анализ методов: собственная CNN vs MediaPipe

Для реализации системы управления роботом жестами мы протестировали два подхода: использование собственной сверточной нейронной сети (CNN) и применение фреймворка MediaPipe. Сравнительный

анализ проводился на наборе из 100 жестовых команд в условиях реального времени. Точность MediaPipe оказалась намного выше, чем у собственной CNN. В данной главе рассмотрим детали эксперимента, причины различий в точности и сильные стороны каждого из методов.

Собственная CNN - 46%, низкая устойчивость к изменению угла обзора, освещения. MediaPipe - 87%, реже ошибался, но иногда путал схожие жесты.

Анализ метода собственной CNN

Преимущества: гибкость - модель была разработана специально под задачу, что позволило учитывать специфику набора данных, контроль над архитектурой модели и процессом обучения.

Недостатки: для обучения CNN использовался относительно небольшой набор данных, что ограничило способность модели к обобщению. Чувствительность к условиям - освещение: модель плохо справлялась с изменениями яркости, угол обзора: если рука находилась под нестандартным углом, точность падала. Медленная разработка: Процесс сбора данных, их разметки и обучения модели занял значительное время.

Возможные причины ошибок: ограничение в размере и разнообразии обучающего набора данных, трудности с выделением ключевых признаков - модель обучалась на сырых изображениях, что требует больше данных и вычислительных ресурсов для точного выделения ключевых признаков.

Анализ метода MediaPipe

Преимущества: MediaPipe предоставляет предобученные модули для выделения ключевых точек руки, что значительно ускоряет разработку, интеграция фильтров для сглаживания координат ключевых точек. Устойчивость к изменениям условий съемки, таким как освещение и угол обзора. Использование ключевых точек вместо анализа всего изображения минимизирует количество ошибок.

Недостатки: MediaPipe предлагает универсальную модель, которая не всегда идеально подходит под узкоспециализированные задачи. Несмотря на доступность базовых решений, глубокая адаптация метода требует

значительных усилий.

Причины большей точности MediaPipe:

MediaPipe извлекает координаты 21 ключевой точки руки, что снижает сложность задачи. Вместо анализа всего изображения модель работает с компактными и информативными признаками. Готовые алгоритмы и встроенные фильтры обеспечивают высокую скорость и устойчивость работы. Модели MediaPipe были обучены на больших объемах разнообразных данных, что улучшает их способность к обобщению.

MediaPipe оказался более точным решением благодаря использованию предобученной модели для выделения ключевых точек и стабилизации данных.

Собственная CNN имеет потенциал для улучшения, однако требует значительно больших усилий для сбора данных, их подготовки и последующей оптимизации.

В условиях ограниченного времени и ресурсов MediaPipe является оптимальным выбором для задач управления жестами. Однако в долгосрочной перспективе разработка собственной модели может быть оправдана для достижения максимально точных результатов в специфических сценариях.

Таким образом, выбор метода зависит от баланса между необходимостью кастомизации и доступными ресурсами. В рамках данного проекта использование MediaPipe позволило достичь высокой точности и сократить время разработки.

Заключение

В ходе выполнения данной работы была разработана система управления роботом с использованием жестов, которая сочетает в себе методы машинного обучения и симуляцию движений на 2D гриде. В результате исследования была реализована классификационная модель на основе сверточной нейронной сети (CNN) для распознавания жестов, а также создана симуляция движения робота, использующая команды, полученные от модели.

Модель, использующая инструменты Mediapipe, показала высокую точность классификации (87%), что подтверждает ее эффективность для распознавания основных жестов, таких как «вперед», «вправо», «влево» и «вниз».

Симуляция движения робота на 2D гриде успешно исполнила полученные команды, что продемонстрировало работоспособность системы в контролируемых условиях.

Тестирование и анализ: в процессе тестирования были выявлены сильные стороны системы, такие как высокая точность распознавания основных жестов, а также ограничения, связанные с точностью распознавания сложных жестов и устойчивостью модели к изменениям условий съемки.

Полученные результаты показывают, что предложенная система имеет высокие перспективы для использования в различных областях, где требуются интуитивно понятные способы управления, включая робототехнику, виртуальную реальность и другие сферы человеко-машинного взаимодействия.

Однако, несмотря на достигнутые результаты, система требует доработки для повышения устойчивости к реальным условиям эксплуатации. В частности, требуется улучшить точность классификации сложных жестов, а также адаптировать модель для работы с реальными роботами, где могут возникать дополнительные сложности, такие как динамические изменения в

окружающей среде или взаимодействие с несколькими объектами одновременно.

Перспективы дальнейшего развития включают внедрение более сложных архитектур для распознавания жестов, интеграцию системы с реальными роботами, а также расширение функционала за счет использования дополнительных сенсоров и улучшения интерфейса взаимодействия с пользователем. В целом, результаты работы открывают новые возможности для создания более гибких и удобных систем управления на основе жестов, что представляет собой важный шаг вперед в области робототехники и взаимодействия человека с машиной.

Список литературы

- 1 Hand gesture recognition method based on HOG-LBP features for mobile devices URL: <https://www.sciencedirect.com/science/article/pii/S1877050918312353>
- 2 A Comprehensive Guide to Convolutional Neural Networks URL: <https://towardsdatascience.com/a-comprehensive-guide-to-convolutional-neural-networks-the-eli5-way-3bd2b1164a53>
- 3 MediaPipe Gesture Recognition URL: https://ai.google.dev/edge/mediapipe/solutions/vision/gesture_recognizer
- 4 MediaPipe: A Framework for Building Perception Pipelines. Camillo Lugaresi, Jiuqiang Tang, Hadon Nash, Chris McClanahan, Esha Uboweja, Michael Hays, Fan Zhang, Chuo-Ling Chang, Ming Guang Yong, Juhyun Lee, Wan-Teh Chang, Wei Hua, Manfred Georg, Matthias Grundmann. International Conference on Inventive Computation Technologies (ICICT), 2019.
- 5 Hand Gesture Recognition Dataset URL: <https://www.kaggle.com/datasets/gti-upm/leapgestrecog>
- 6 *MobileNetV2: Inverted Residuals and Linear Bottlenecks*, статья. Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., & Chen, L. 2018. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR).

Приложение А

Листинг кода

```
mp_hands = mp.solutions.hands
mp_drawing = mp.solutions.drawing_utils
mp_holistic = mp.solutions.holistic
grid_size = 5
robot_position = [2, 2]

# функция движения робота
def move_robot(gesture):
    global robot_position

    x, y = robot_position

    if gesture == "UP":
        x -= 1
    elif gesture == "DOWN":
        x += 1
    elif gesture == "LEFT":
        y -= 1
    elif gesture == "RIGHT":
        y += 1

    # ограничения по сетке
    x = max(0, min(x, grid_size - 1))
    y = max(0, min(y, grid_size - 1))
    robot_position = (x, y)

# построение 2д сетки
```

```

def display_grid():
    grid = np.zeros((grid_size, grid_size))
    grid[robot_position[0], robot_position[1]] = 1

plt.imshow(grid, cmap='cool', interpolation='nearest')
plt.xticks(ticks=range(grid_size), labels=range(grid_size))
plt.yticks(ticks=range(grid_size), labels=range(grid_size))
plt.title("Robot on 2D Grid")
plt.show()def take_photo_and_move_robot_with_mediapipe(filename='photo.jpg',
quality=0.8):
    js = Javascript("""
        async function takePhoto(quality) {
            const div = document.createElement('div');
            const capture = document.createElement('button');
            capture.textContent = 'Capture';
            div.appendChild(capture);

            const video = document.createElement('video');
            video.style.display = 'block';
            const stream = await navigator.mediaDevices.getUserMedia({video: true});

            document.body.appendChild(div);
            div.appendChild(video);
            video.srcObject = stream;
            await video.play();

            // Resize the output to fit the video element.
            google.colab.output.setIframeHeight(document.documentElement.scrollHeight, true);

            // Wait for Capture to be clicked.
            await new Promise((resolve) => capture.onclick = resolve);

            const canvas = document.createElement('canvas');
            canvas.width = video.videoWidth;

```



```

        canvas.height = video.videoHeight;
        canvas.getContext('2d').drawImage(video, 0, 0);
        stream.getVideoTracks()[0].stop();
        div.remove();
        return canvas.toDataURL('image/jpeg', quality);
    }
    ""
    display(js)

```

try:

```

# Capture photo from webcam
data = eval_js('takePhoto({})'.format(quality))
img = js_to_image(data)

# Recognize gesture using MediaPipe
gesture = recognize_gesture_with_mediapipe(img)

# Move the robot based on the recognized gesture
move_robot(gesture)

```

```

# Annotate the image with the gesture
annotated_img = cv2.putText(
    img, f"Gesture: {gesture}", (50, 50),
    cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2, cv2.LINE_AA
)
cv2.imwrite(filename, annotated_img)

```

```

# Display the updated grid
display_grid()

```

```

# Return the annotated image
display(Image(filename))

```

```

except Exception as err:
    print(f"Error: {err}")

```

Приложение Б

Листинг кода

```
kaggle datasets download gti-upm/leapgestrecog
unzip /content/leapgestrecog.zip

import numpy as np
import pandas as pd
import os
import cv2

import tensorflow as tf
from sklearn.model_selection import train_test_split
from tensorflow.keras.preprocessing.image import ImageDataGenerator

from tensorflow import keras
from keras.models import Sequential
from keras.layers import Dense, Dropout, Flatten
from keras.layers import Conv2D, MaxPooling2D

import matplotlib.pyplot as plt
import torch

# If there's a GPU available...
if torch.cuda.is_available():

    # Tell PyTorch to use the GPU.
    device = torch.device("cuda")

    print('There are %d GPU(s) available.' % torch.cuda.device_count())

    print('We will use the GPU:', torch.cuda.get_device_name(0))

# If not...
else:
    print('No GPU available, using the CPU instead.')
```

```

device = torch.device("cpu")
dir = '/content/leapGestRecog'
images = []
labels = []
for directory in os.listdir(dir):
    for subDir in os.listdir(os.path.join(dir,directory)):
        for img in os.listdir(os.path.join(dir, directory, subDir)):
            img_path = os.path.join(dir, directory, subDir, img)
            images.append(img_path)
            labels.append(subDir)
Iseries = pd.Series(images, name="Images")
Lseries = pd.Series(labels, name="labels")
hand_gesture_data = pd.concat([Iseries, Lseries], axis=1)
hand_gesture_df = pd.DataFrame(hand_gesture_data)
print(hand_gesture_df.head())
pd.Series(labels).value_counts()
X_train, X_test = train_test_split(hand_gesture_df, test_size=0.2, random_state=42)
train_set, val_set = train_test_split(hand_gesture_df, test_size=0.3, random_state=42)
image_gen = ImageDataGenerator(preprocessing_function=
tf.keras.applications.mobilenet_v2.preprocess_input)
train = image_gen.flow_from_dataframe(dataframe=
train_set,x_col="Images",y_col="labels",
                                     target_size=(244,244),
                                     color_mode='rgb',
                                     class_mode="categorical",
                                     batch_size=4,
                                     shuffle=False
                                     )
test = image_gen.flow_from_dataframe(dataframe= X_test,x_col="Images",
y_col="labels",
                                     target_size=(244,244),
                                     color_mode='rgb',
                                     class_mode="categorical",
                                     batch_size=4,
                                     shuffle= False

```

```

        )

    val_generator = image_gen.flow_from_dataframe(dataframe=val_set, x_col="Images",
        y_col="labels",
        target_size=(244,244),
        color_mode='rgb',
        class_mode="categorical",
        batch_size=4,
        shuffle=False
    )

classes=list(train.class_indices.keys())
print (classes)
def show_hand_gesture(image_gen):
    test_dict = test.class_indices
    classes = list(test_dict.keys())
    images, labels=next(image_gen)
    plt.figure(figsize=(20,20))
    length = len(labels)
    if length<25:
        r=length
    else:
        r=25
    for i in range(r):
        plt.subplot(5,5,i+1)
        image=(images[i]+1)/2
        plt.imshow(image)
        index=np.argmax(labels[i])
        class_name=classes[index]
        plt.title(class_name, color="green",fontsize=16)
        plt.axis('off')
    plt.show()
show_hand_gesture(train)
model = keras.models.Sequential([
    keras.layers.Conv2D(filters=128, kernel_size=(8, 8), strides=(3, 3), activation='relu',
input_shape=(224, 224, 3)),

```

```

keras.layers.BatchNormalization(),

keras.layers.Conv2D(filters=256, kernel_size=(5, 5), strides=(1, 1), activation='relu',
padding="same"),
keras.layers.BatchNormalization(),
keras.layers.MaxPool2D(pool_size=(3, 3)),

keras.layers.Conv2D(filters=256, kernel_size=(3, 3), strides=(1, 1), activation='relu',
padding="same"),
keras.layers.BatchNormalization(),
keras.layers.Conv2D(filters=256, kernel_size=(1, 1), strides=(1, 1), activation='relu',
padding="same"),
keras.layers.BatchNormalization(),
keras.layers.Conv2D(filters=256, kernel_size=(1, 1), strides=(1, 1), activation='relu',
padding="same"),
keras.layers.BatchNormalization(),

keras.layers.Conv2D(filters=512, kernel_size=(3, 3), activation='relu',
padding="same"),
keras.layers.BatchNormalization(),
keras.layers.MaxPool2D(pool_size=(2, 2)),

keras.layers.Conv2D(filters=512, kernel_size=(3, 3), activation='relu',
padding="same"),
keras.layers.BatchNormalization(),

keras.layers.Conv2D(filters=512, kernel_size=(3, 3), activation='relu',
padding="same"),
keras.layers.BatchNormalization(),

keras.layers.MaxPool2D(pool_size=(2, 2)),

keras.layers.Conv2D(filters=512, kernel_size=(3, 3), activation='relu',
padding="same"),
keras.layers.BatchNormalization(),

```

```

keras.layers.MaxPool2D(pool_size=(2, 2)),

keras.layers.Flatten(),
keras.layers.Dense(1024, activation='relu'),
keras.layers.Dropout(0.5),
keras.layers.Dense(1024, activation='relu'),
keras.layers.Dropout(0.5),
keras.layers.Dense(10, activation='softmax')
])

model.compile(
    loss='categorical_crossentropy',
    optimizer=tf.optimizers.SGD(learning_rate=0.001),
    metrics=['accuracy']
)

model.summary()
from keras.utils import plot_model

plot_model(model, to_file='model_plot.png', show_shapes=True,
show_layer_names=True)

history = model.fit(train, epochs=3, validation_data=val, verbose=1)
model.evaluate(test, verbose=1)
model.save("hand_gesture_Model.h5")
pred = model.predict(test)
pred = np.argmax(pred, axis=1)

labels = (train.class_indices)
labels = dict((v,k) for k,v in labels.items())
pred2 = [labels[k] for k in pred]
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('model accuracy')
plt.ylabel('accuracy')

```

```

plt.xlabel('epoch')
plt.legend(['train', 'val'], loc='upper left')
plt.show()
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('model loss')
plt.ylabel('loss')
plt.xlabel('epoch')
plt.legend(['train', 'val'], loc='upper left')
plt.show()

from sklearn.metrics import confusion_matrix, accuracy_score, classification_report

y_test = X_test.labels
print(classification_report(y_test, pred2))
print("Accuracy of the Model:", "{:.1f}%".format(accuracy_score(y_test, pred2)*100))
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix

class_labels = ['Palm', 'I', 'Fist', 'Fist_moved', 'Thumb', 'Index', 'Ok', 'Palm_moved', 'C',
'Down']

cm = confusion_matrix(y_test, pred2)

plt.figure(figsize=(10, 5))
sns.heatmap(cm, annot=True, fmt='g', vmin=0, cmap='Blues')

plt.xticks(ticks=[0.5, 1.5, 2.5, 3.5, 4.5, 5.5, 6.5, 7.5, 8.5, 9.5], labels=class_labels,
rotation=25)
plt.yticks(ticks=[0.5, 1.5, 2.5, 3.5, 4.5, 5.5, 6.5, 7.5, 8.5, 9.5], labels=class_labels,
rotation=0)
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()

```